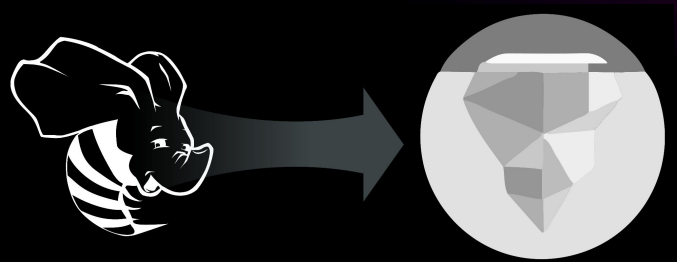
Starburst

Migrating Hive to Iceberg
Starburst Workshops

A hands-on webinar
with Lester Martin from DevRel



Connection before content

Lester Martin – https://linktr.ee/lestermartin

- Developer Relations @ Starburst
  - Blogging & forums
  - Webinars & videos
  - User groups & events
  - Training & tutorials
- 30+ years of technology experience
  - Started journey on TRS-80 Model III
  - Played most roles, but a programmer at my core
  - ½ career in OLTP and ½ in data analytics
  - Decade+ of "big data" experience to include
    - Trino/Starburst, Hadoop, Hive, Spark
    - NiFi, Kafka, Storm, Flink
    - HBase, MongoDB

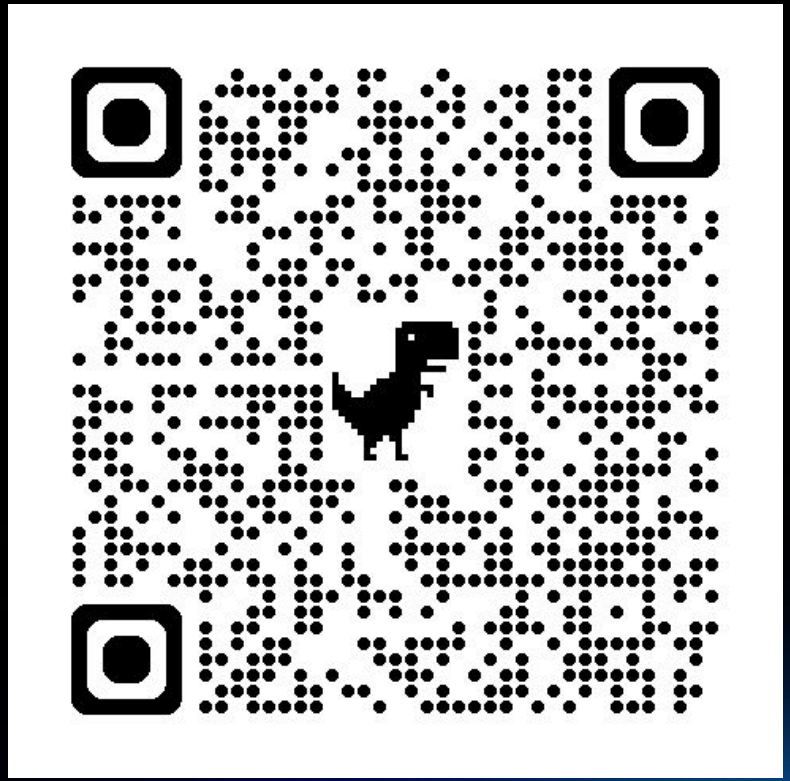
devrel@starburst.io

Starburst

Environment setup

Setup and instructions located on Github

https://github.com/starburstdata/devrel/blob/main/workshops/iceberg-migration/INSTRUCTIONS.md



Environment

Using Starburst Galaxy and S3

Exercises

All SQL needed avail for download

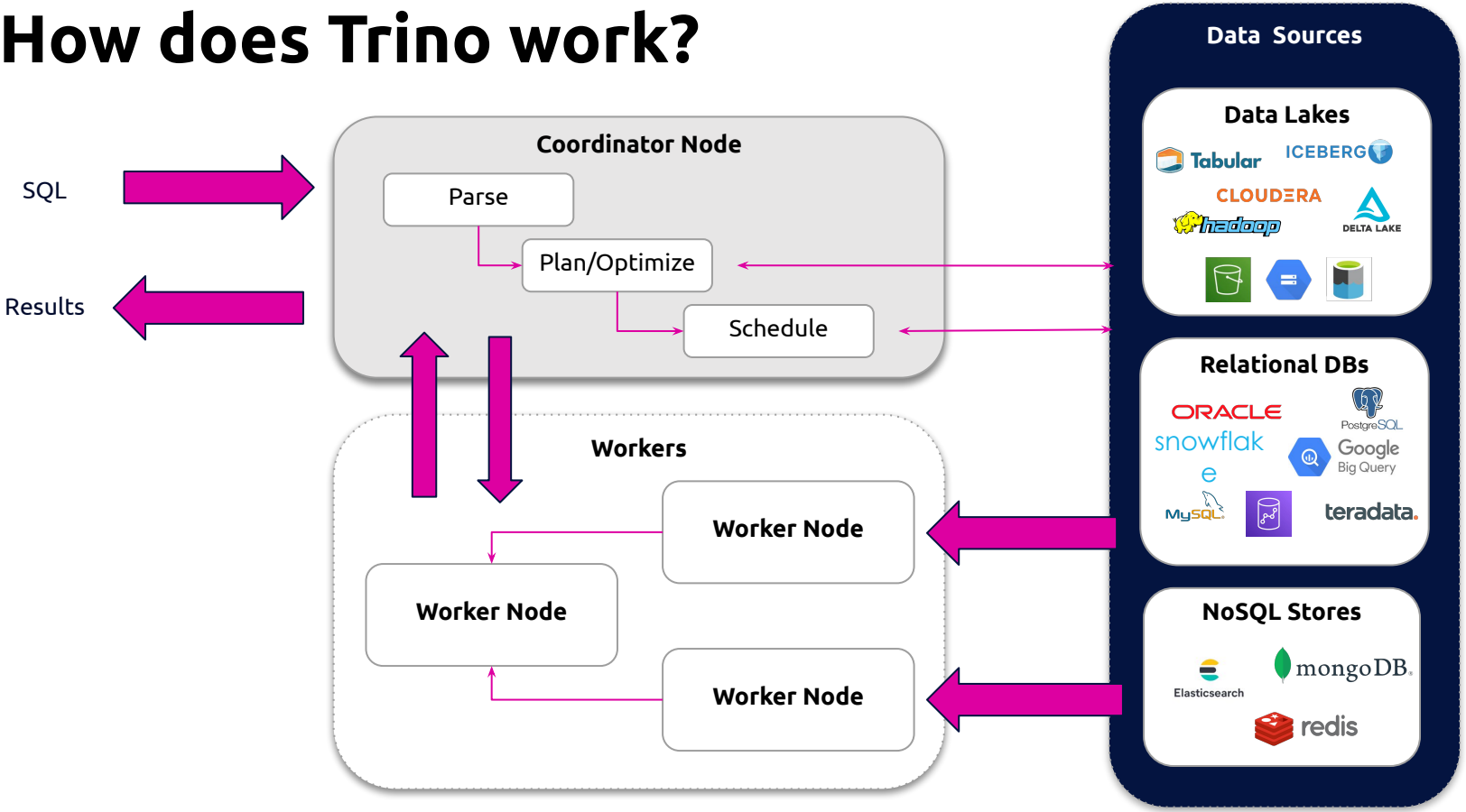
Starburst

Hands-on

Validate env setup

Iceberg + Trino = Icehouse

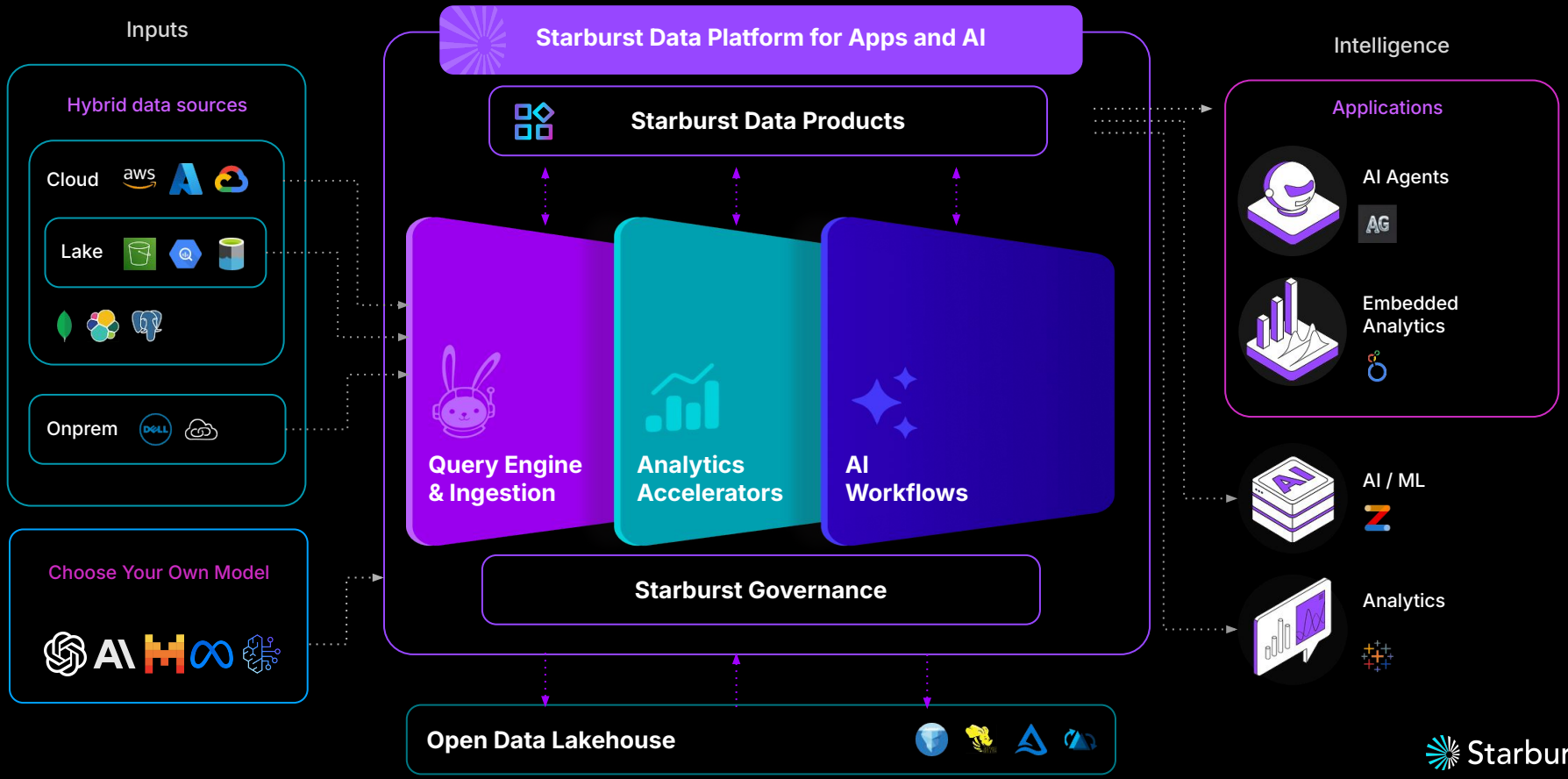
How does Trino work?



https://trino.io

Starburst

The Starburst Platform

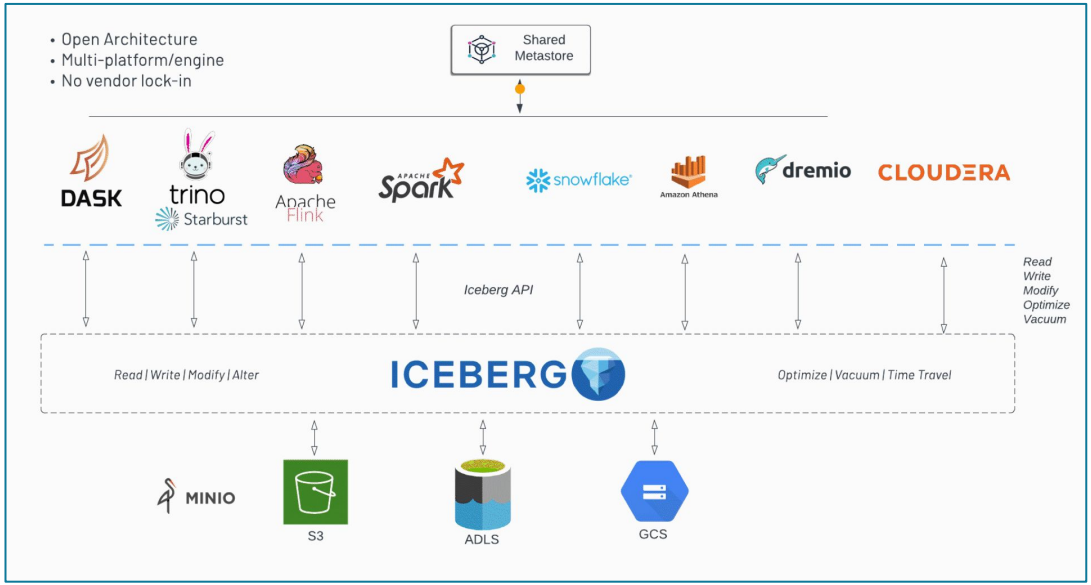


Starburst

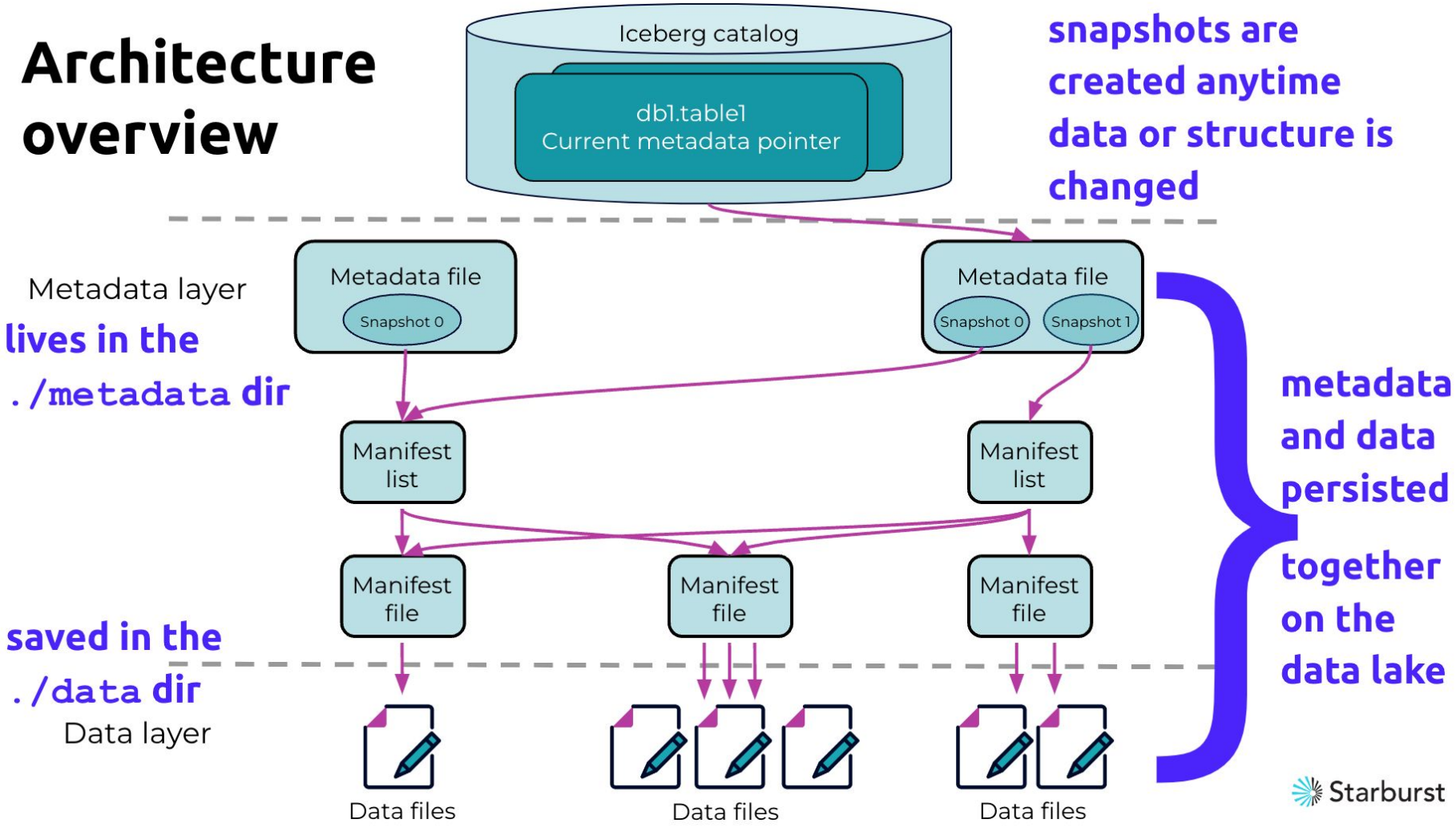
Apache Iceberg

- Created by Ryan Blue & Daniel Weeks at Netflix in 2017
- Solve the challenges of performance, data modification and schema evolution in the lake + offer benefits of versioning
- Uses open data concepts (orc, parquet, avro) and architecture

Multi-Engine Platform



Architecture overview



Starburst fits well in an Open Hybrid data stack

Global federated access to 50+ data sources beyond the lake

Starburst fits into *any* environment

Common start: migrate select workloads

Open table formats

And/or use Starburst to launch delayed projects

Open file formats

Many retain mixed architectures long-term

Commodity on-premise and cloud storage and compute

Starburst

Flexible consumption models

Starburst Enterprise

Deploy anywhere

Enterprise-grade software backed by professional support and services

On prem, in the cloud, hybrid, and multi-cloud

Starburst Galaxy

Built for the cloud

Fully managed cloud data lake analytics built and supported by the creators of Trino

Available on leading public clouds

Available via marketplaces

Starburst

Hands-on

Versioning w/Iceberg

When NOT to migrate from Hive

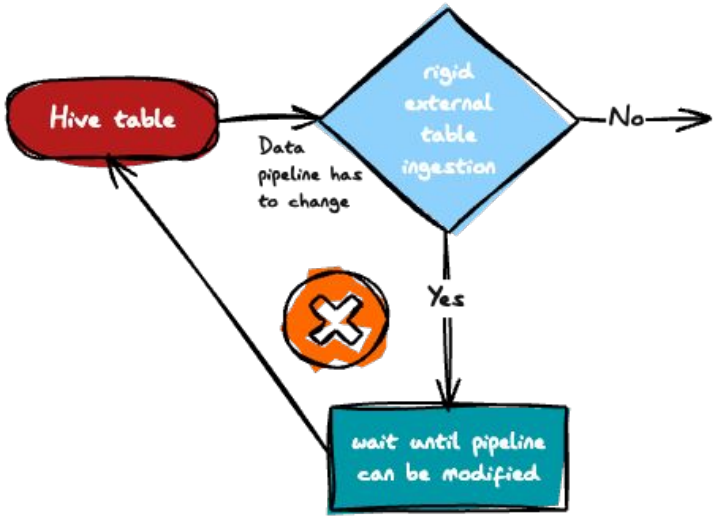
Reasons to NOT migrate

Time and effort will be required - *make sure the juice is worth the squeeze*

Reasons to NOT migrate

Time and effort will be required - *make sure the juice is worth the squeeze*

- **File add process requires modification** - usually associated w/external tables



Reasons to NOT migrate

Time and effort will be required - *make sure the juice is worth the squeeze*

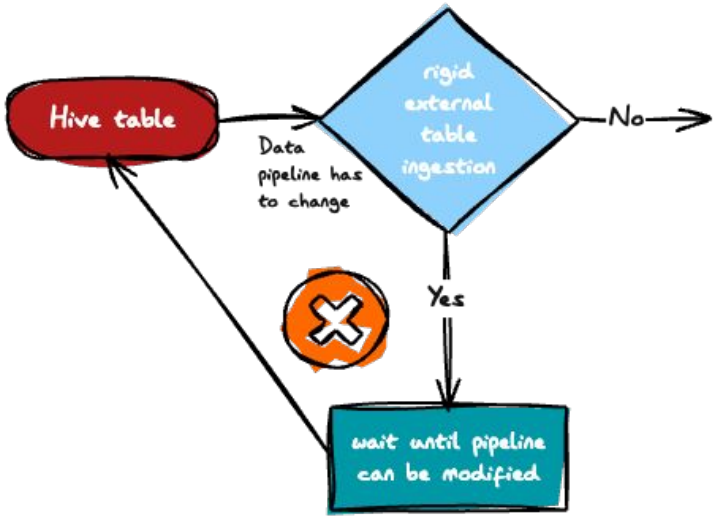
- **File add process requires modification** - usually associated w/external tables

NOTE: Move to `add_files` procedure

The `add_files` procedure supports adding files, and therefore the contained data, to a target table, specified after `ALTER TABLE`. It loads the files from a object storage path specified with the required `location` parameter. The files must use the specified `format`, with `ORC` and `PARQUET` as valid values. The target Iceberg table must use the same format as the added files. The procedure does not validate file schemas for compatibility with the target Iceberg table. The `location` property is supported for partitioned tables.

The following examples copy `ORC`-format files from the location `s3://my-bucket/a/path` into the Iceberg table `iceberg_customer_orders` in the `lakehouse` schema of the `example` catalog:

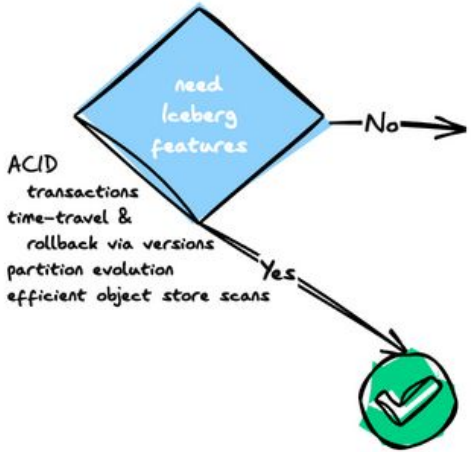
```
ALTER TABLE example.lakehouse.iceberg_customer_orders
EXECUTE add_files(
    location => 's3://my-bucket/a/path',
    format => 'ORC')
```



Reasons to NOT migrate

Time and effort will be required - *make sure the juice is worth the squeeze*

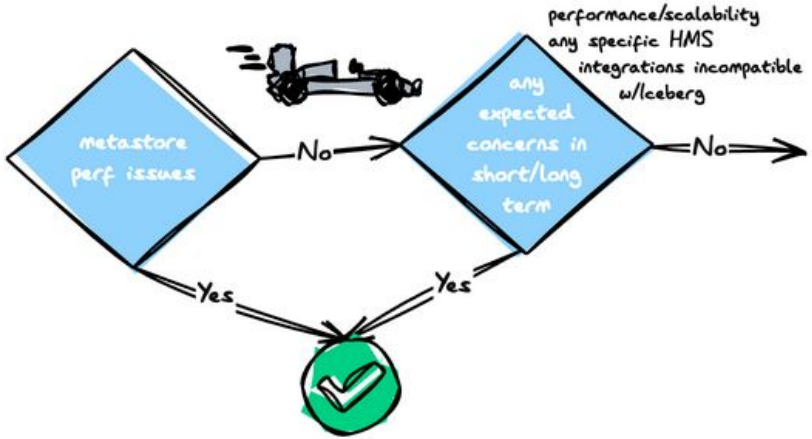
- **File add process requires modification** - usually associated w/external tables
- **Functional benefits not valuable enough** - ACID transactions, versioning, hidden partitioning, schema/partition evolution



Reasons to NOT migrate

Time and effort will be required - *make sure the juice is worth the squeeze*

- **File add process requires modification** - usually associated w/external tables
- **Functional benefits not valuable enough** - ACID transactions, versioning, hidden partitioning, schema/partition evolution
- **High degree of confidence in existing performance and scalability** - testing validates no foreseeable concerns in the short to long term



Reasons to NOT migrate

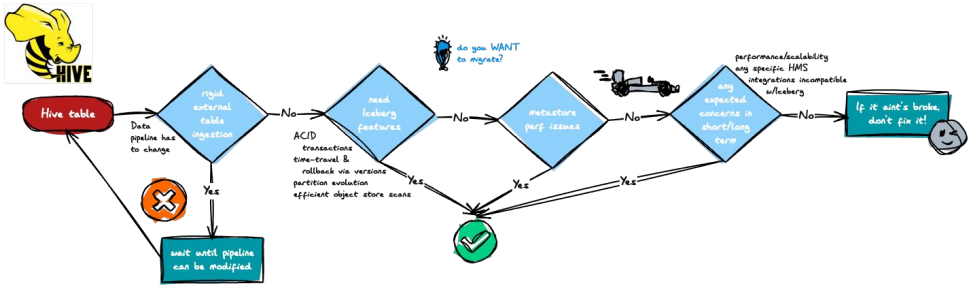
Time and effort will be required - *make sure the juice is worth the squeeze*

- **File add process requires modification** - usually associated w/external tables
- **Functional benefits not valuable enough** - ACID transactions, versioning, hidden partitioning, schema/partition evolution
- **High degree of confidence in existing performance and scalability** - testing validates no foreseeable concerns in the short to long term
- **Simply no resources available to migrate** - much less the necessary testing to find any unforeseen issues

Reasons to NOT migrate

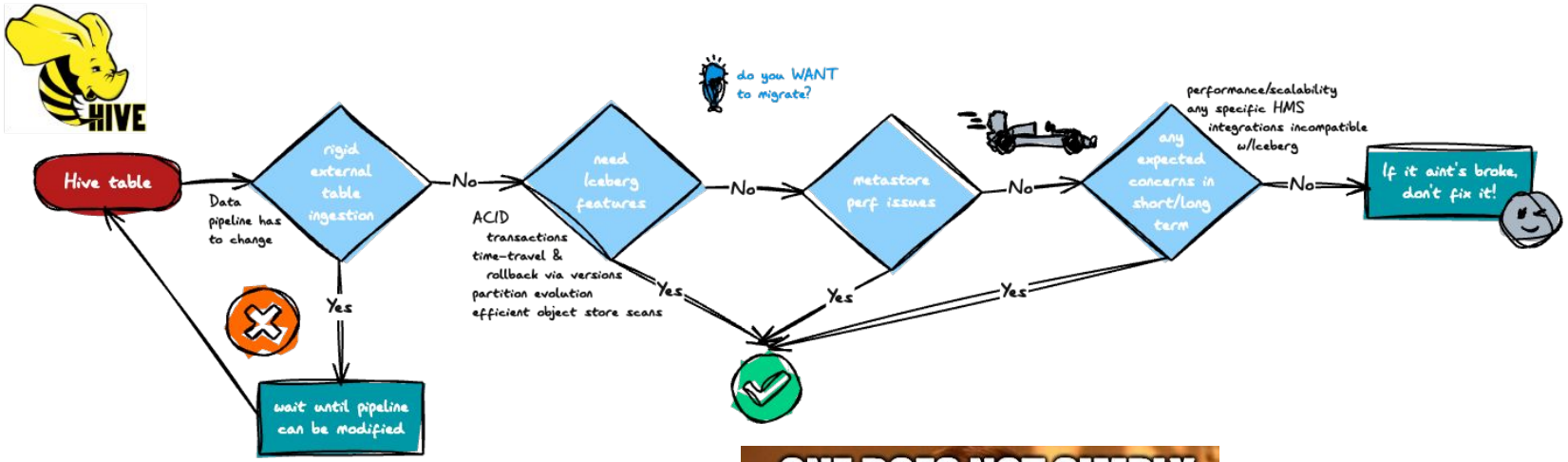
Time and effort will be required - *make sure the juice is worth the squeeze*

- **File add process requires modification** - usually associated w/external tables
- **Functional benefits not valuable enough** - ACID transactions, versioning, hidden partitioning, schema/partition evolution
- **High degree of confidence in existing performance and scalability** - testing validates no foreseeable concerns in the short to long term
- **Simply no resources available to migrate** - much less the necessary testing to find any unforeseen issues



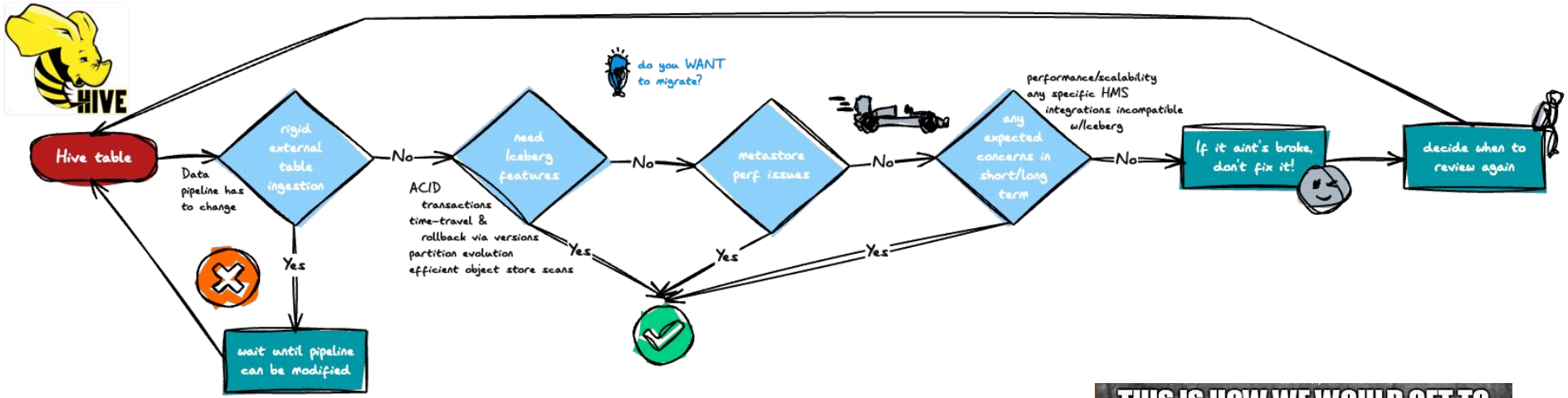
Reasons to NOT migrate

Time and effort will be required - *make sure the juice is worth the squeeze*



Reasons to NOT migrate (or not yet)

Time and effort will be required - *make sure the juice is worth the squeeze*



Migration strategies

Migration strategies

Two approaches - *let's define them*

Shadow migration process

Creates a new Iceberg table modeled after the original Hive table whose values are then inserted into the new table; the original table can then be dropped

The in-place method

Avoids rewriting the data files by modifying the table format type in the catalog and only building additional Iceberg metadata files

In-place migration requirements

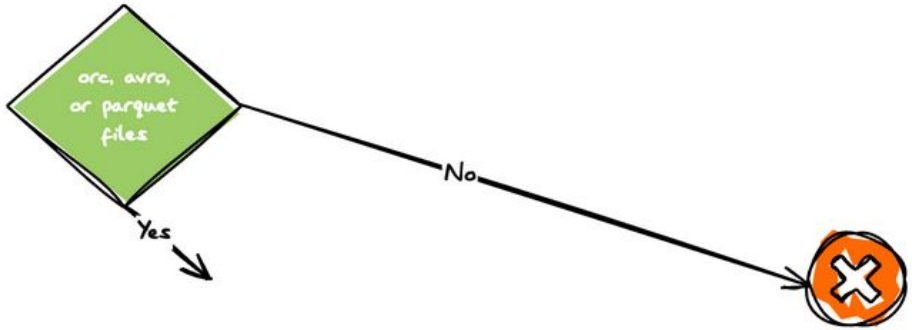
Takes over table's folder & creates Iceberg metadata - *recycles files*

NOTE: External tables convert to managed tables

In-place migration requirements

Takes over table's folder & creates Iceberg metadata - *recycles files*

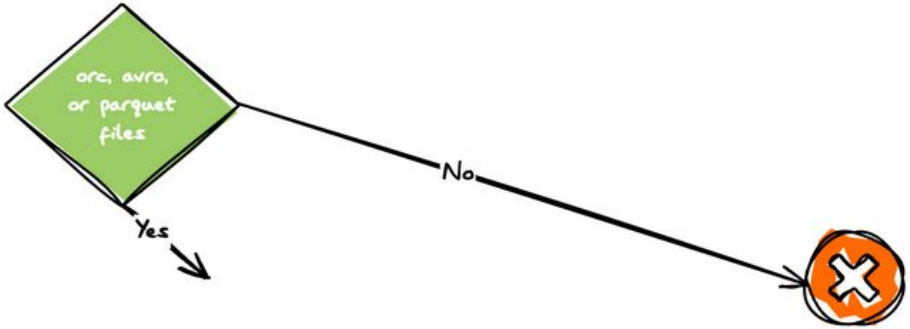
- **Compatible data file formats** - Parquet, ORC & Avro



In-place migration requirements

Takes over table's folder & creates Iceberg metadata - *recycles files*

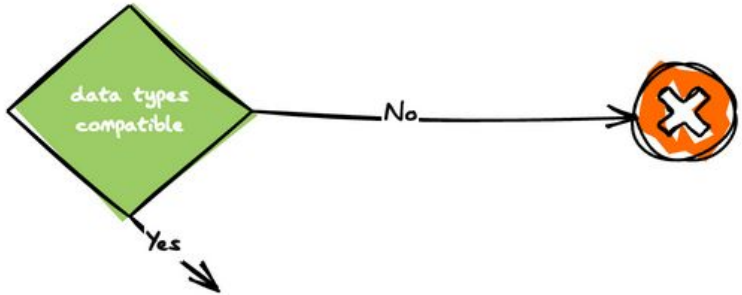
- **Compatible data file formats** - Parquet, ORC & Avro



In-place migration requirements

Takes over table's folder & creates Iceberg metadata - *recycles files*

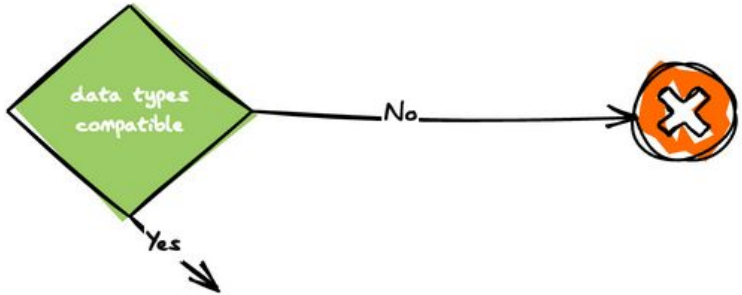
- **Compatible data file formats** - Parquet, ORC & Avro
- **Compatible column data types** - ex: `CHAR` and `TINYINT` not supported and `TIMESTAMP` is not compatible (millisecond vs microsecond precision)



In-place migration requirements

Takes over table's folder & creates Iceberg metadata - *recycles files*

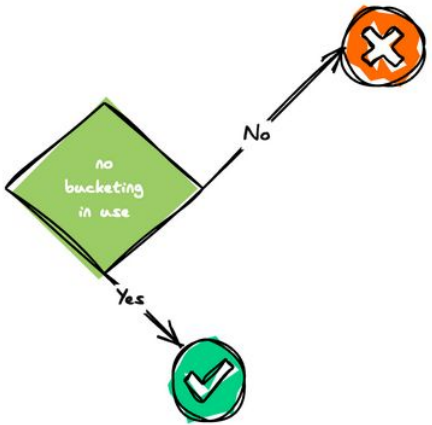
- **Compatible data file formats** - Parquet, ORC & Avro
- **Compatible column data types** - ex: `CHAR` and `TINYINT` not supported and `TIMESTAMP` is not compatible (millisecond vs microsecond precision)



In-place migration requirements

Takes over table's folder & creates Iceberg metadata - *recycles files*

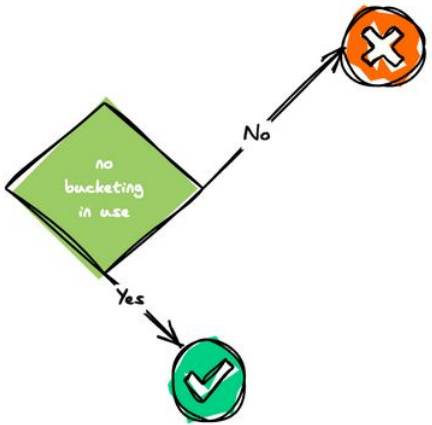
- **Compatible data file formats** - Parquet, ORC & Avro
- **Compatible column data types** - ex: `CHAR` and `TINYINT` not supported and `TIMESTAMP` is not compatible (millisecond vs microsecond precision)
- **`bucketed_by/bucket_count` are ignored** - Iceberg implements bucketing as numbered partitioned folders, not files



In-place migration requirements

Takes over table's folder & creates Iceberg metadata - *recycles files*

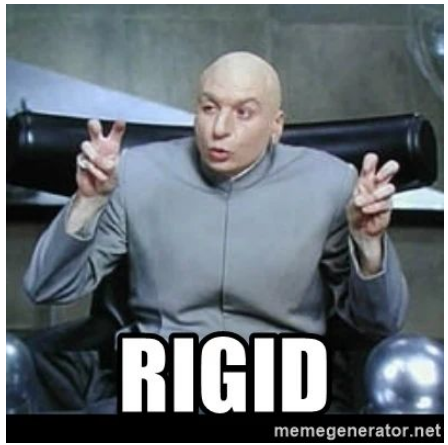
- **Compatible data file formats** - Parquet, ORC & Avro
- **Compatible column data types** - ex: `CHAR` and `TINYINT` not supported and `TIMESTAMP` is not compatible (millisecond vs microsecond precision)
- **`bucketed_by`/`bucket_count` are ignored** - Iceberg implements bucketing as numbered partitioned folders, not files



In-place migration requirements

Takes over table's folder & creates Iceberg metadata - *recycles files*

- **Compatible data file formats** - Parquet, ORC & Avro
- **Compatible column data types** - ex: `CHAR` and `TINYINT` not supported and `TIMESTAMP` is not compatible (millisecond vs microsecond precision)
- **`bucketed_by`/`bucket_count` are ignored** - Iceberg implements bucketing as numbered partitioned folders, not files



In-place migration requirements

Takes over table's folder & creates Iceberg metadata - *recycles files*

- **Compatible data file formats** - Parquet, ORC & Avro
- **Compatible column data types** - ex: `CHAR` and `TINYINT` not supported and `TIMESTAMP` is not compatible (millisecond vs microsecond precision)
- **`bucketed_by/bucket_count` are ignored** - Iceberg implements bucketing as numbered partitioned folders, not files

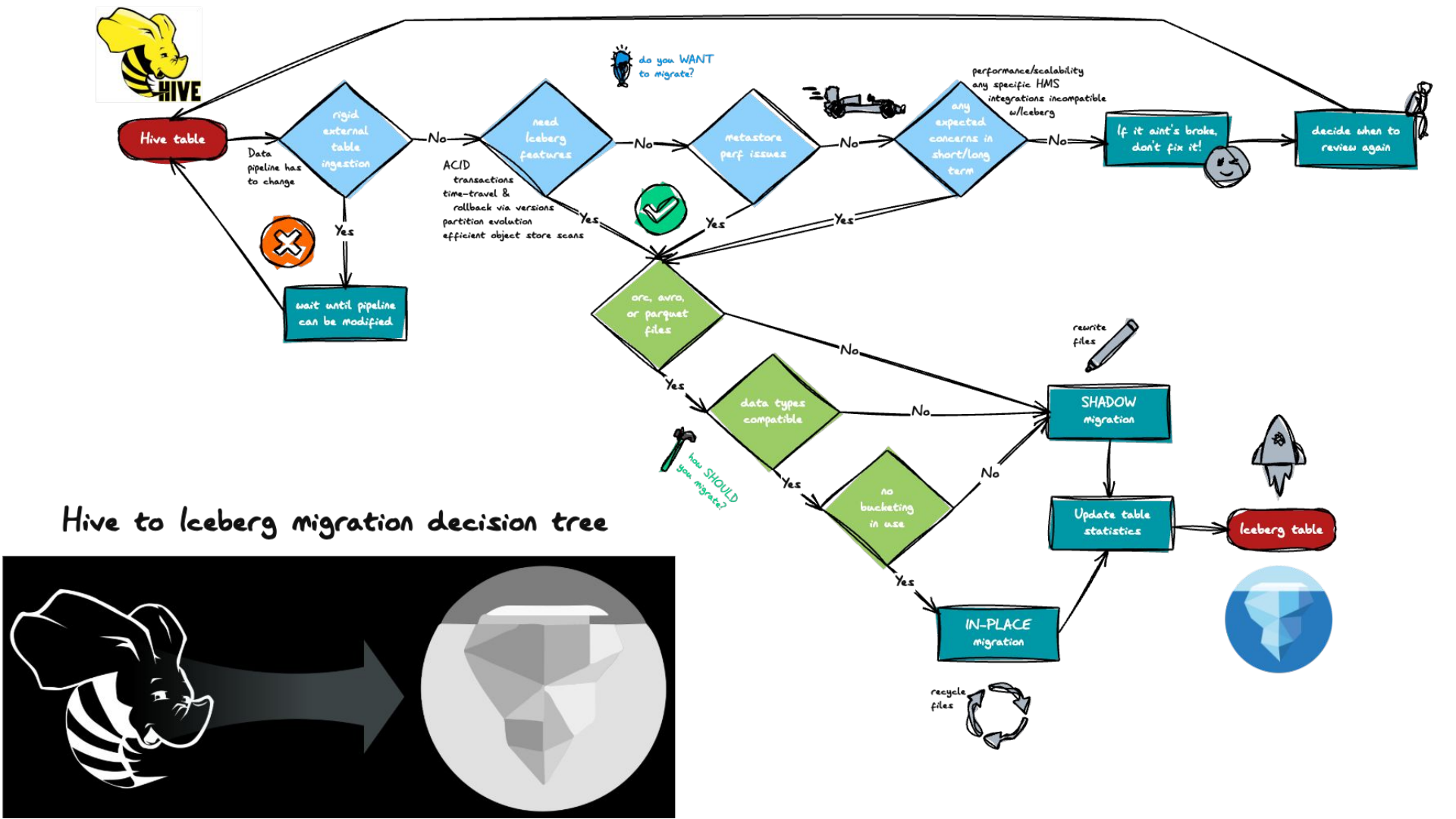
In-place migration requirements

Takes over table's folder & creates Iceberg metadata - *recycles files*

- **Compatible data file formats** - Parquet, ORC & Avro
- **Compatible column data types** - ex: `CHAR` and `TINYINT` not supported and `TIMESTAMP` is not compatible (millisecond vs microsecond precision)
- **`bucketed_by`/`bucket_count` are ignored** - Iceberg implements bucketing as numbered partitioned folders, not files

Put it all together

Starburst



Hive to Iceberg migration decision tree

Additional considerations

Migration considerations

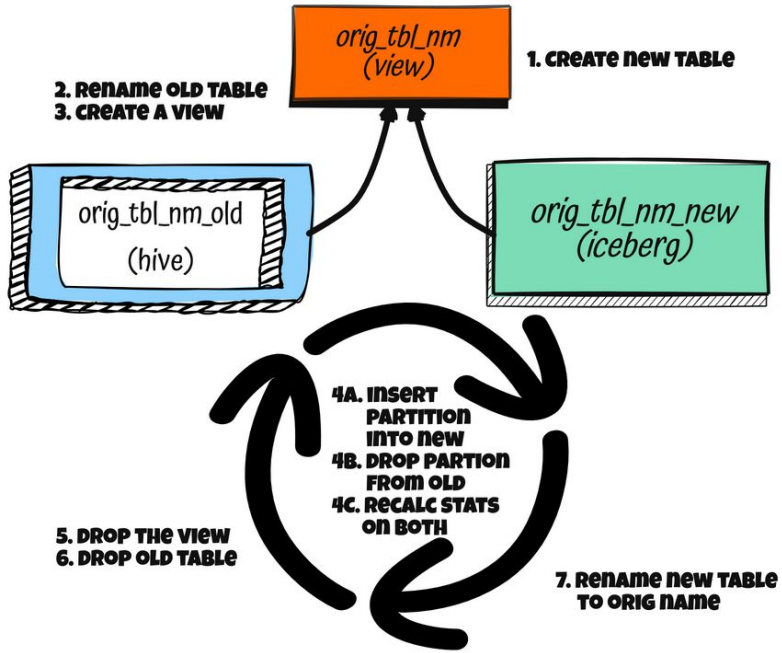
Any migration is an major event and should be planned & tested accordingly

- Test, test, and more test - *don't forget about a backout strategy*
- Automate maintenance activities
- Consider staging rewrites for very large, heavily-partitioned, tables

Migration considerations

Any migration is an major event and should be planned & tested accordingly

- Test, test, and more test - *don't forget about a backout strategy*
- Automate maintenance activities
- Consider staging rewrites for very large, heavily-partitioned, tables

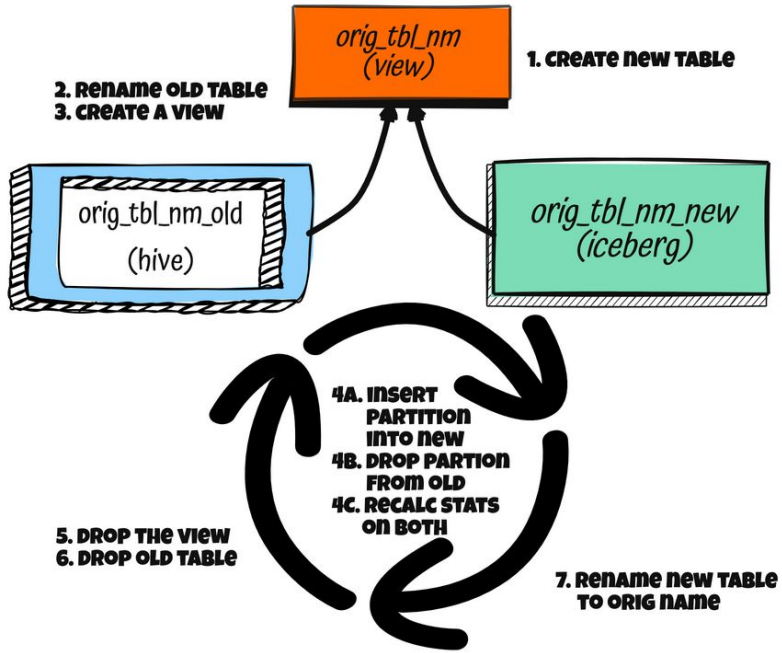


Migration considerations

Any migration is an major event and should be planned & tested accordingly

- Test, test, and more test - *don't forget about a backout strategy*
- Automate maintenance activities
- Consider staging rewrites for very large, heavily-partitioned, tables

If time permits



Next steps

What are my next steps?

Decide if you are ready to begin & then engage Starburst for help

- Evaluate your existing data lake tables
- Consider tactical focus on largest tables vs comprehensive migration of all
- Visit https://www.starburst.io/solutions/data-migrations/hive-iceberg/ for more information
- Get free guidance on your Hive to Iceberg migration by providing contact info at https://www.starburst.io/info/hive-to-iceberg-migration-guidance/

Demo artifacts at https://github.com/starburstdata/devrel/tree/main/workshops/iceberg-migration

Starburst

Thank you!


Starburst